
POZNAŃ UNIVERSITY OF TECHNOLOGY

FACULTY OF CONTROL, ROBOTICS
AND ELECTRICAL ENGINEERING

INSTITUTE OF ROBOTICS AND MACHINE INTELLIGENCE

DIVISION OF CONTROL AND INDUSTRIAL ELECTRONICS



Light Intensity Control System

MICROPROCESSOR SYSTEMS

LABORATORY REPORT

MICHAŁ PIETRZAK, 155315

MICHAL.PIETRZAK@STUDENT.PUT.POZNAN.PL

PIOTR TRUSIEWICZ 155322

PIOTR.TRUSIEWICZ@STUDENT.PUT.POZNAN.PL

INSTRUCTOR:

ADRIAN WÓJCIK, M.Sc.

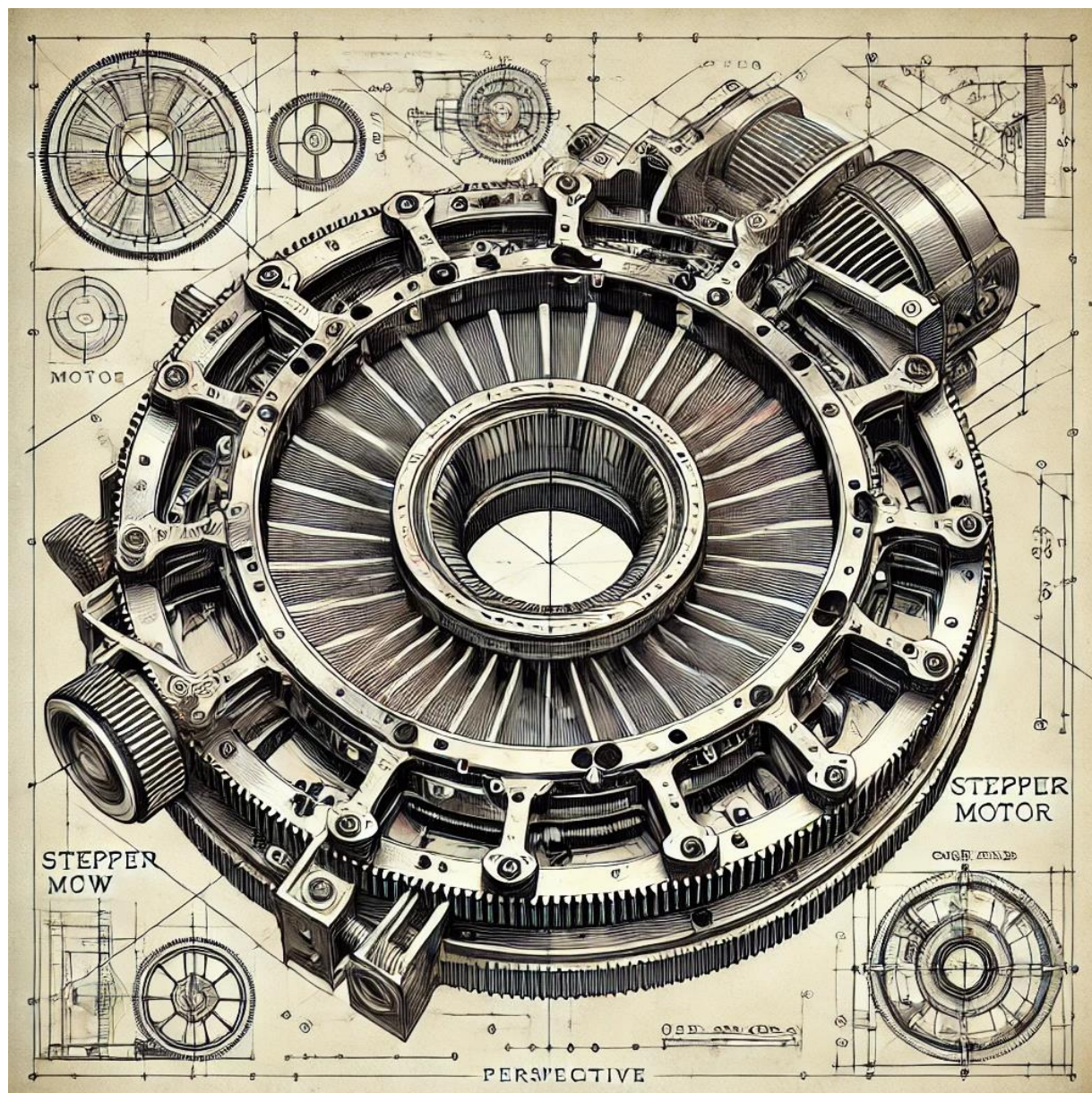
ADRIAN.WOJCIK@PUT.POZNAN.PL

13-12-2024



Contents

1. Introduction and purpose.....	4
2. Model 3D.....	4
3. Systems.....	7
4. Performance	11
5. References & Sources	16



1. INTRODUCTION AND PURPOSE

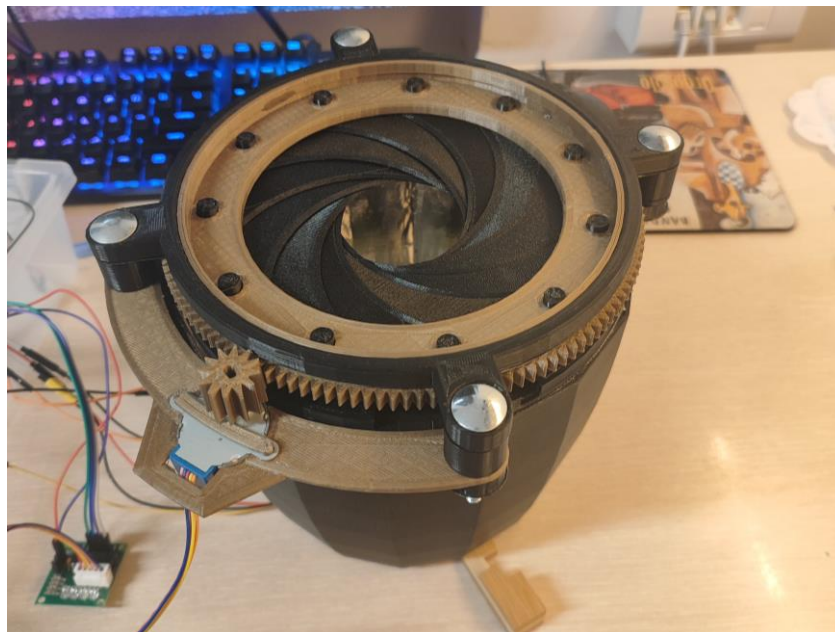
The goal of our project was to create a system that controls the amount of light inside a chamber where a sensor is placed. We opted for a mechanical solution inspired by the human eye—designing a movable iris that smoothly adjusts the amount of incoming light.

The system is powered by a stepper motor, ensuring precise and fast regulation of the aperture. The entire setup is controlled by a Nucleo L476RG board, with the desired value adjustable via a rotary knob. Real-time readings from both the sensor and the knob are displayed on an LCD screen.

Since the ambient lighting conditions may be insufficient, we integrated an LED to illuminate the inside of the chamber. This LED also allows for fine-tuning the light intensity to achieve the exact lumen value when the iris mechanism alone is not enough.

Our project has a wide range of applications. It can regulate the amount of light delivered to environments such as greenhouses and can also serve as an educational tool for learning control systems.

Photo 1. Final photo of the mechanism



2. MODEL 3D

The 3D model of our project was based on an existing design found on Thingiverse. It underwent a series of modifications and improvements to achieve better performance. We added a stepper motor mount, a more precise gear, and redesigned the iris blades, which were trimmed towards the shorter edge to enhance their functionality.



Photo 2. Ray-traced main 3D model



Photo 3. Labeled components of the main model

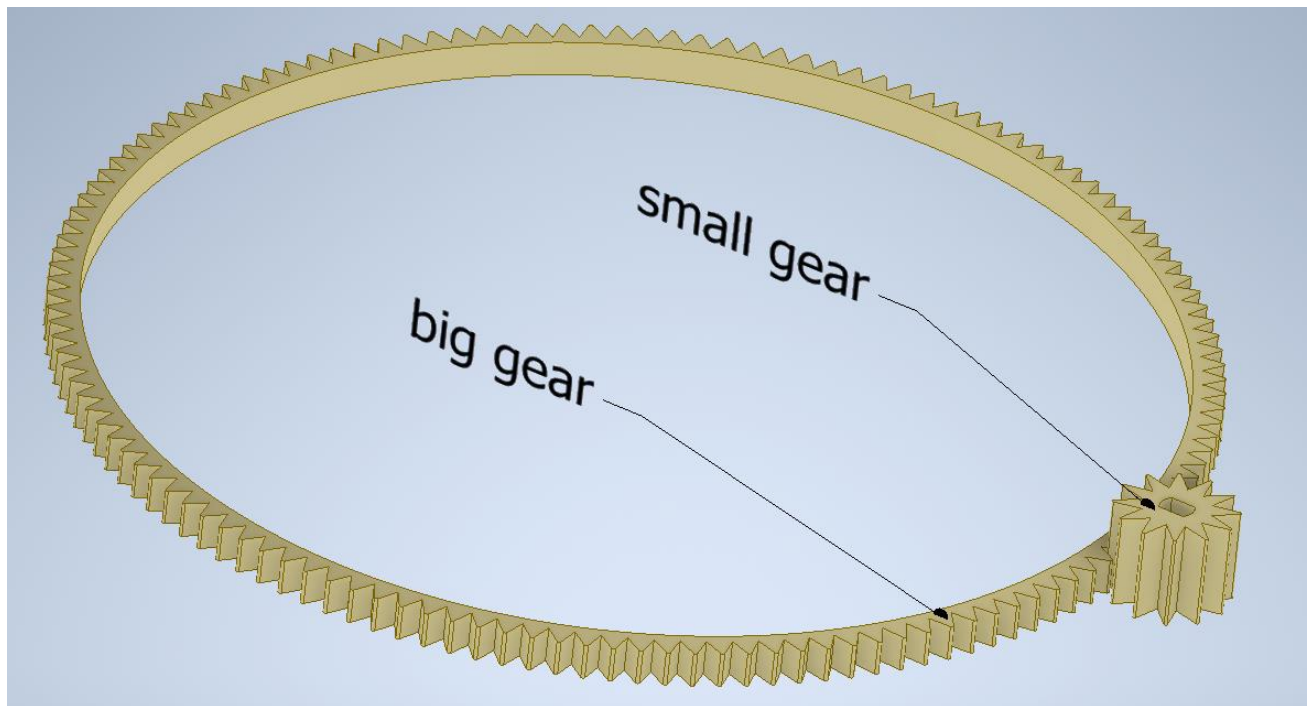


Photo 4. Gears transferring motion from the stepper motor to the pivot. The gear ratio is 12:135.

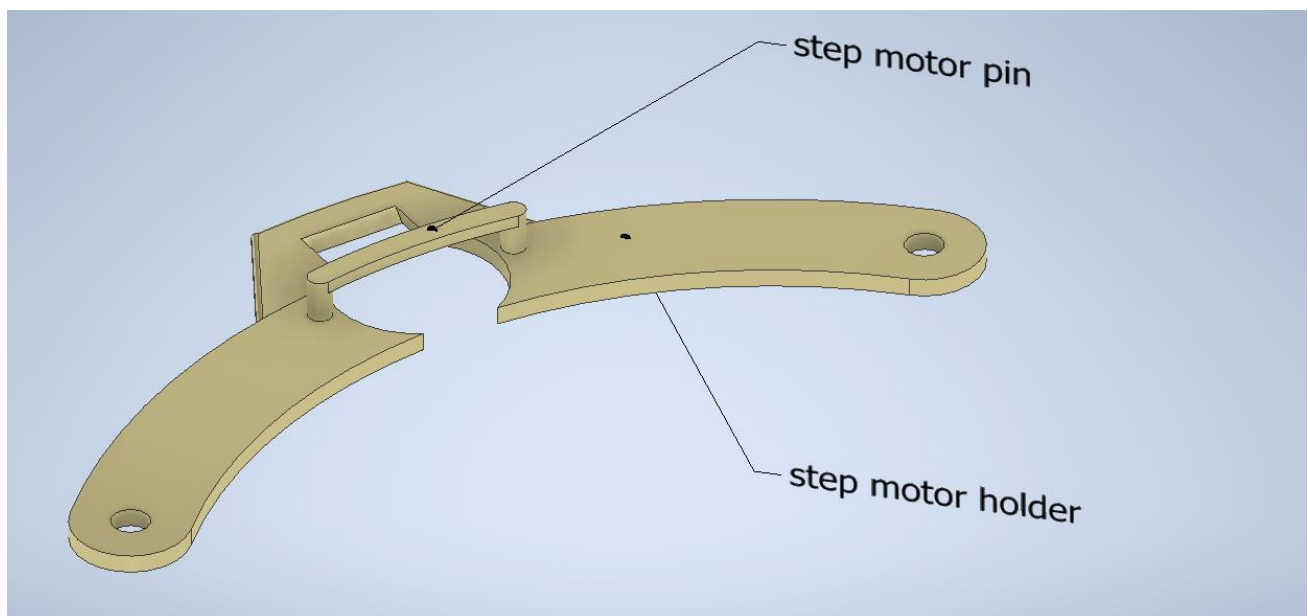
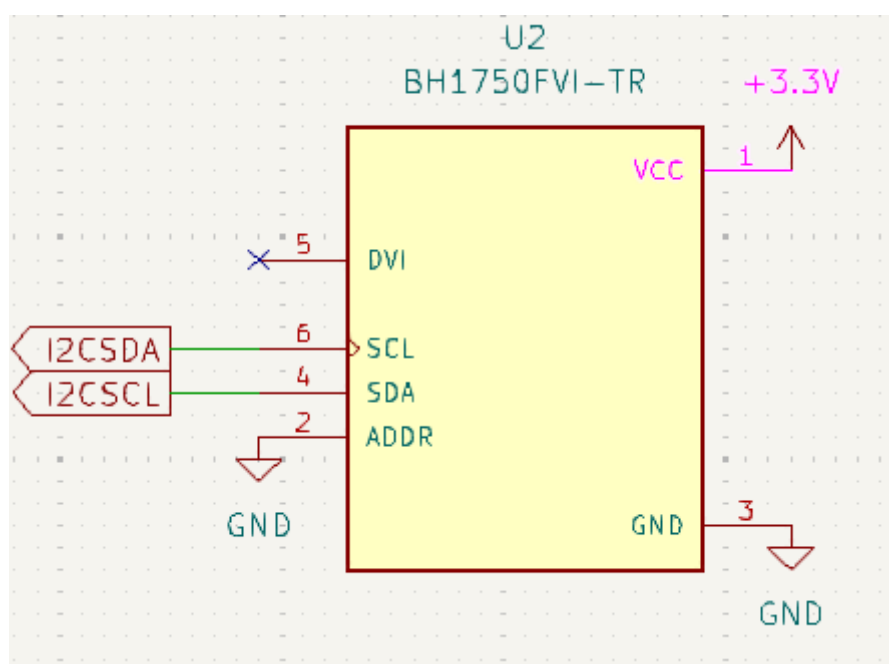


Photo 5. Mount for the stepper motor with a retaining pin.

3. SYSTEMS

1. Light Sensor BH1750

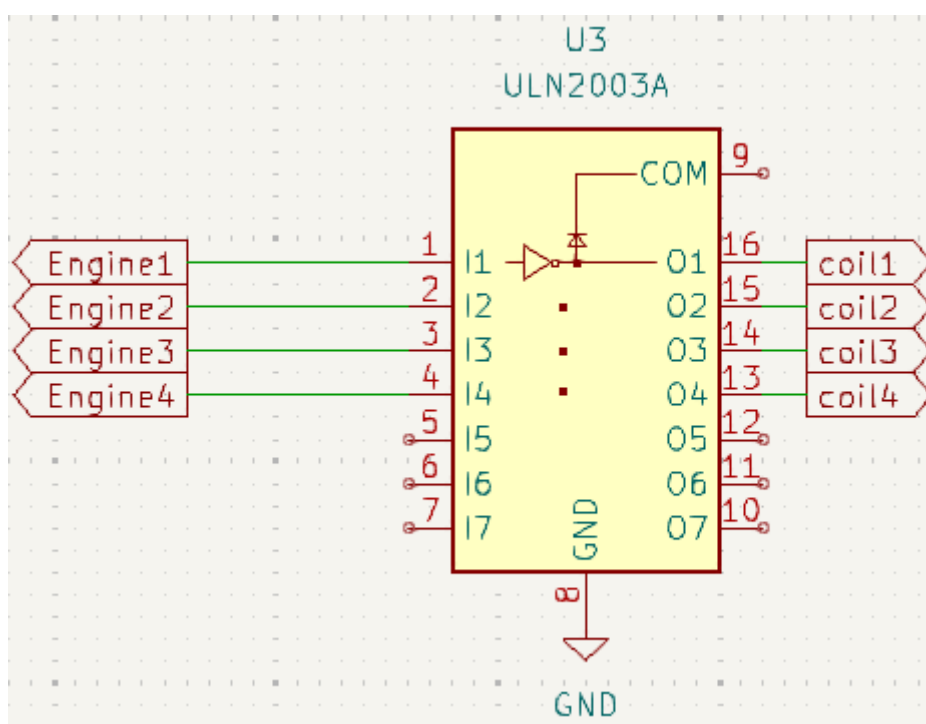
The sensor measures the light intensity inside the chamber and transmits the data to the microcontroller, enabling precise light regulation. It is a digital light intensity sensor operating in the range of 1 - 65,535 lx, with a resolution of 1 or 4 lx depending on the selected mode. It communicates via the I2C (TWI) interface using two lines: data (SDA) and clock (SCL).



Scheme 1. Connection of the light sensor BH1750

2. Stepper motor with gearbox 28BYJ-48 with ULN2003 .

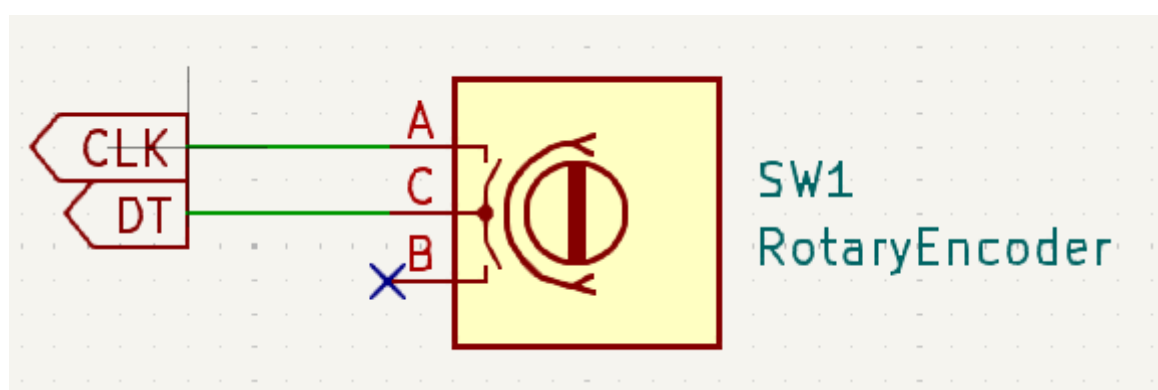
The stepper motor with a built-in 1:64 gearbox, powered by 5V and controlled by a dedicated module. It features a 4-phase step, a 5 mm shaft diameter (without flats), and 35 mm mounting hole spacing. It precisely controls the iris opening by converting electrical pulses into rotational motion. The built-in gearbox increases torque, ensuring smooth and accurate regulation of light inside the chamber.



Scheme 2. Stepper motor connection with ULN2003, which is connected to the step motor.

3. Encoder - Iduino SE055

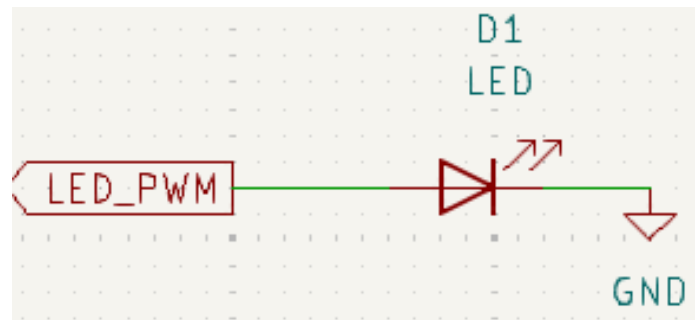
A rotary knob in the form of an incremental encoder, enabling the creation of a simple and intuitive user interface. The module generates a digital signal on the **CLK** and **DT** pins, depending on the direction of rotation. It operates at **3.3V** and allows for precise control of system parameters.



Scheme 3. Connection of encoder – Iduino SE05

4. Diode LED

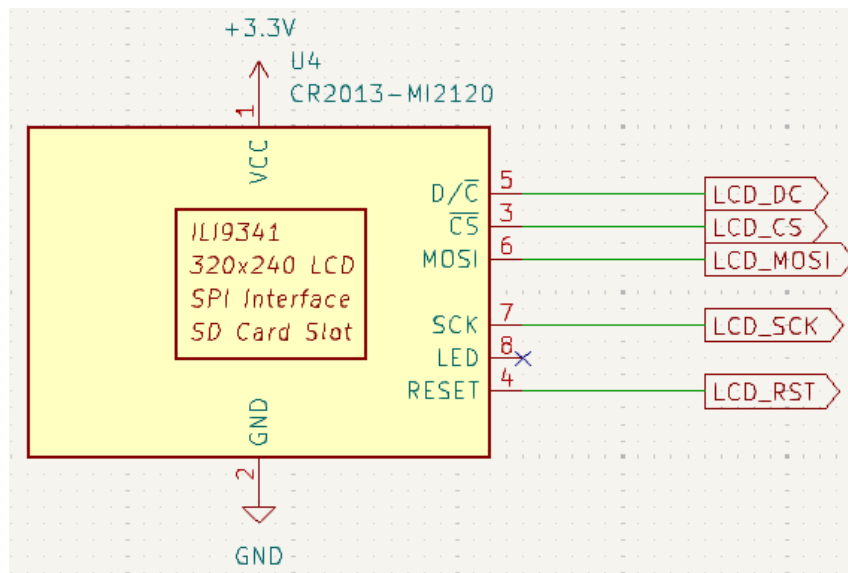
Simple white diode.



Scheme 4. diode LED connection

5. LCD

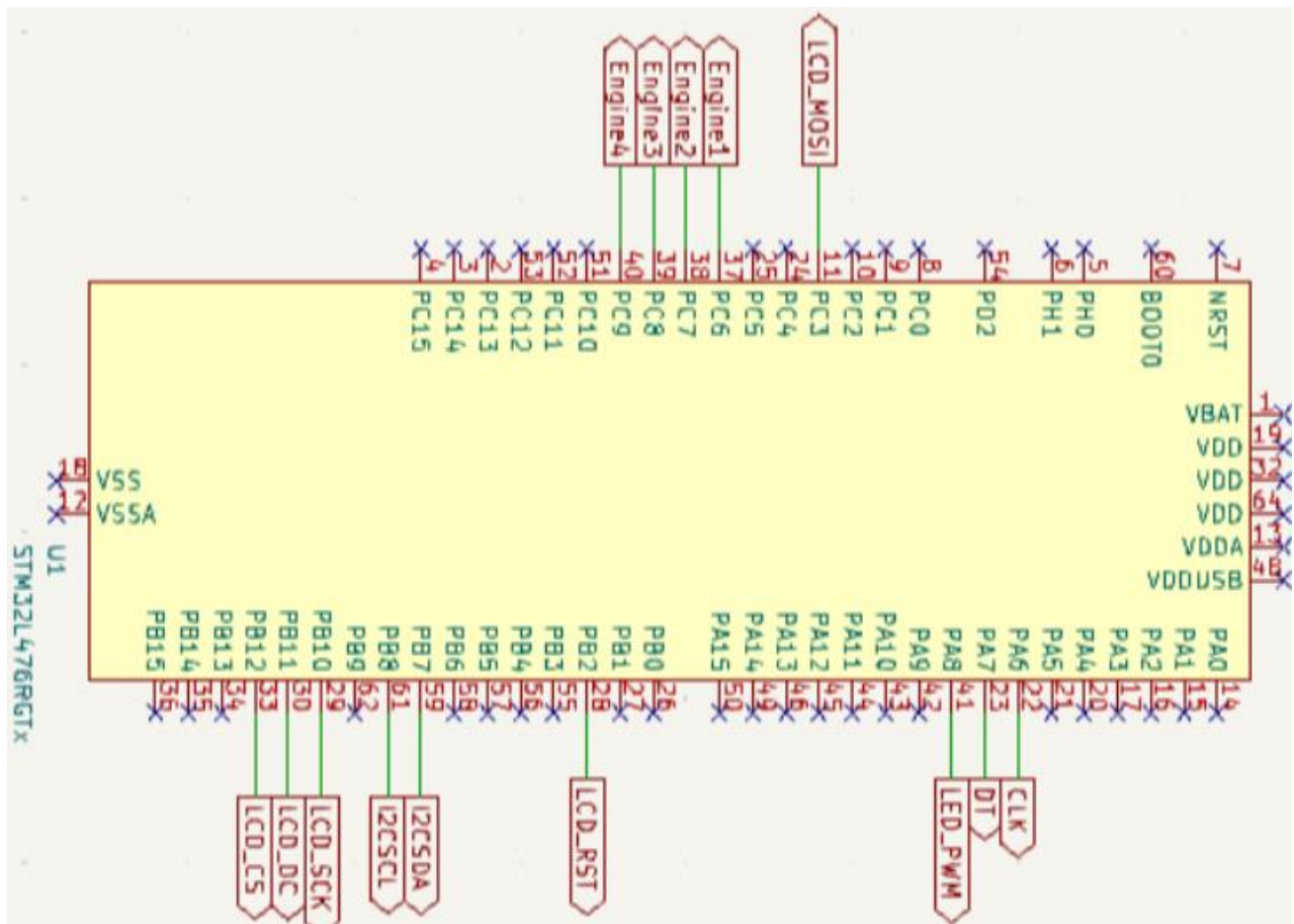
A **1.8"** color **LCD TFT** display with a resolution of **128 x 160 px**. Based on the **ST7735S** driver, it operates at **3.3V** and communicates via **SPI**. It displays the values set by the encoder and the measured light intensity from the sensor. The CS line is active low.



Scheme 5. LCD's connection

6. Nucleo L476RG

The Nucleo module features a 32-bit STM32L476RG microcontroller, based on an ARM Cortex-M4 core running at 80 MHz. It includes 1 MB of Flash memory and 128 kB of RAM. The board is mbed-compatible and equipped with connectors for Arduino Shield add-ons. It supports popular development environments such as Keil, IAR, and GCC-based toolchains. The module also features a built-in ST-Link debugger/programmer, making programming and system diagnostics easier.



Scheme 6. Connection of the Nucleo L476RG

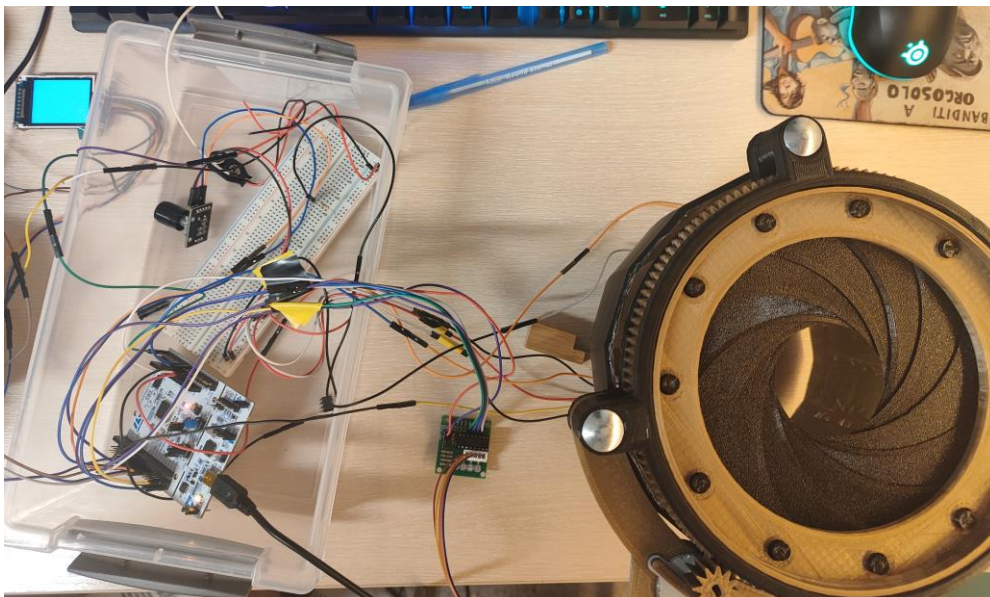


Photo 6. Working circuit, nucleo L476RG and mechanical iris

4. PERFORMANCE

All graphs show the measured sensor value (blue line) and the set value (red line) as a function of time (horizontal axis). The sampling time is 0.5s.

1. As we can see in the diagram, when the LED was operating with the I-controller, we achieved significant gains, but some overshoots were observed. Additionally, certain steady-state disturbances appeared, and we encountered difficulties in eliminating them.

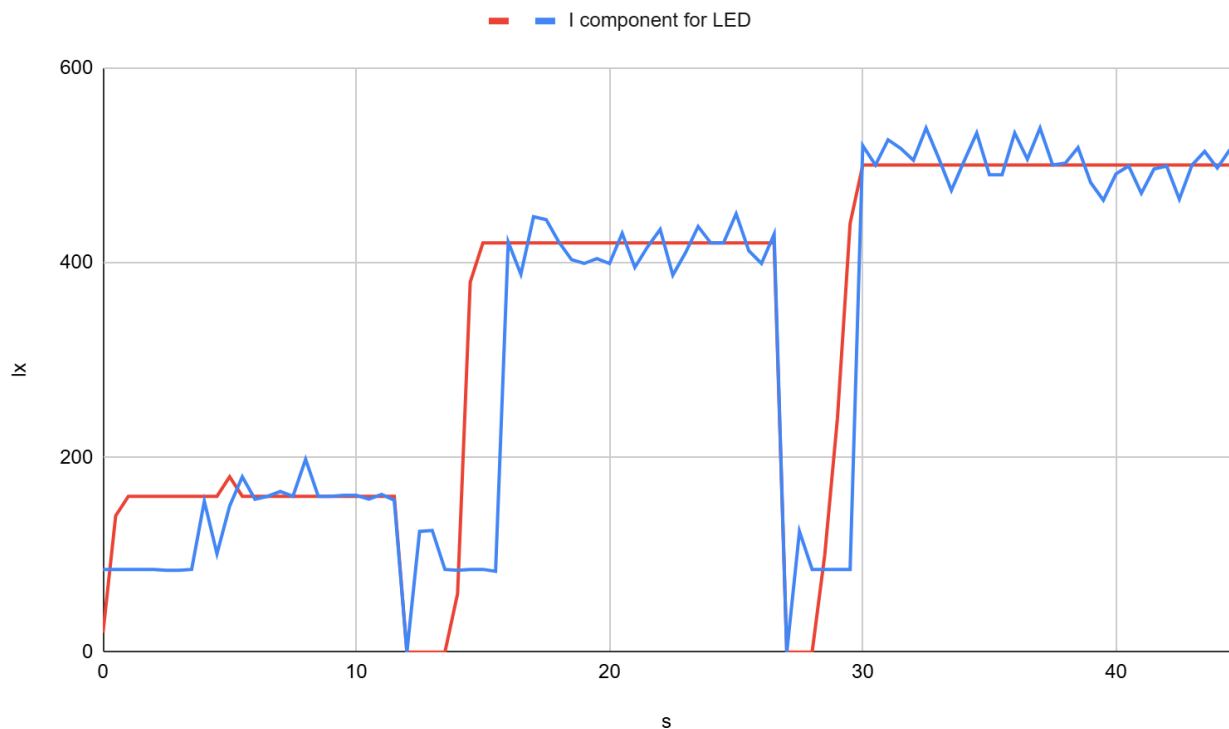


Diagram 1.

```

1. int light = 0;
2. int value = 0;
3. int mode = 0;
4. int pulse = 0;
5. uint8_t ster = 0; // Control variable (1 = diode, 0 = stepper motor)
6. uint8_t engineR = 0; // Stepper motor control (1 = closing, 0 = opening)
7. float currentAngle = 0; // Stepper motor current position
8. int maxAngle = 875; // Max rotation angle
9. int minAngle = 20; // Min rotation angle
10. // PID constants (tune these values)
11. float Kp = 0;
12. float Ki = 5.3;
13. float Kd = 0;
14. // PID variables
15. float error = 0;
16. float previous_error = 0;

```

```
17. float integral = 0;
18. float derivative = 0;
19. float output = 0;
20. float dt = 0.5; // Sampling time in seconds
21.
22. void updatePID() {
23. // Calculate error
24. error = value - light;
25.
26. // Update integral with anti-windup
27. integral += error * dt;
28. if (integral > INTEGRAL_MAX) integral = INTEGRAL_MAX;
29. if (integral < INTEGRAL_MIN) integral = INTEGRAL_MIN;
30.
31. // Calculate derivative
32. derivative = (error - previous_error) / dt;
33.
34. // Compute PID output
35. output = (Kp * error) + (Ki * integral) + (Kd * derivative);
36.
37. // Clamp output to valid PWM range
38. if (output > PWM_MAX) output = PWM_MAX;
39. if (output < PWM_MIN) output = PWM_MIN;
40.
41. // Store error for next iteration
42. previous_error = error;
43. }
44.
45. void control_logic() {
46. updatePID();
47. TIM1->CCR1 = (uint32_t)output; // Set PWM output
48. // } else {
49. // Control stepper motor based on light
50. if (light >= value) {
51. engineR = 1; // Close
52. } else {
53. engineR = 0; // Open
54. }
55.
56. // Stepper motor movement with angle constraints
57. if (engineR == 1 && currentAngle <= maxAngle) {
58. stepper_step_angle(STEPPER_STEP_SIZE, 0, 12);
59. currentAngle += STEPPER_STEP_SIZE;
60. } else if (engineR == 0 && currentAngle >= minAngle) {
61. stepper_step_angle(STEPPER_STEP_SIZE, 1, 12);
62. currentAngle -= STEPPER_STEP_SIZE; }
63. while (1) {
64. control_logic();
65. HAL_Delay(500);}
```

When we shut down the control of the shutter and controlled only the LED using `if()`, we achieved smaller steady-state disturbances and almost no overshoot. However, the gains were quite small, which can be especially seen in the third part of the graph when the set value was 240. The LED took a significant amount of time to achieve enough brightness to reach this level inside our tube.

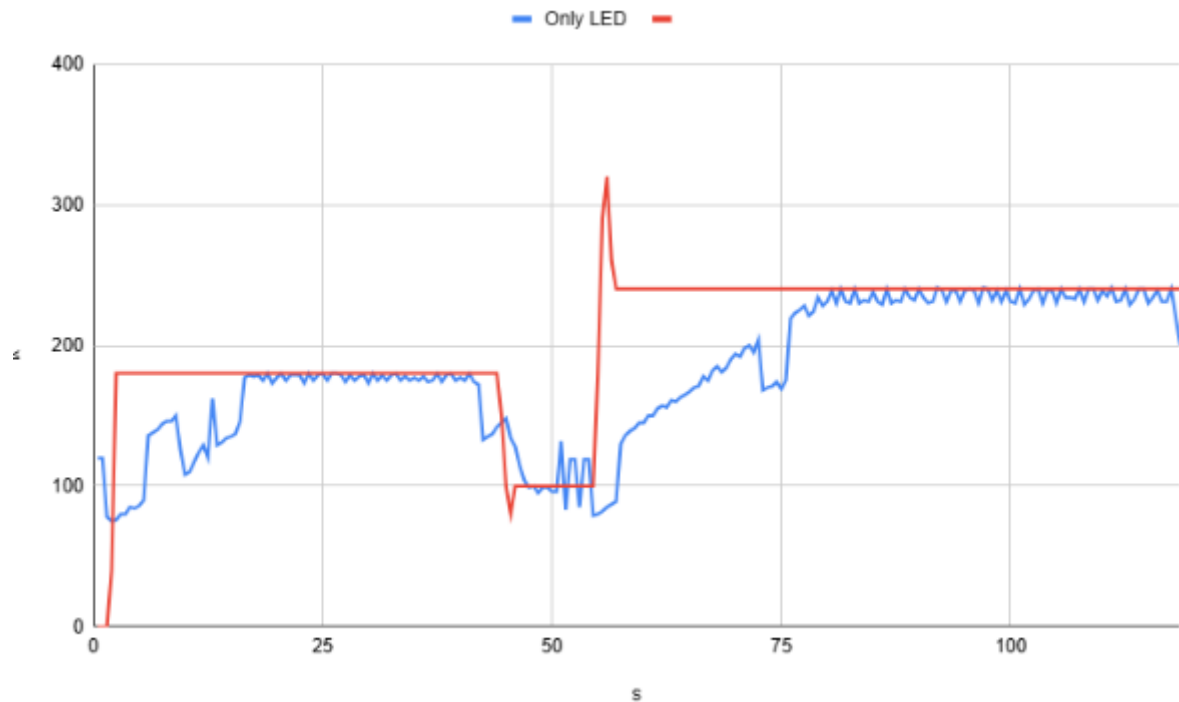


Diagram 2.

```

While(1) {HAL_Delay(200); if (mode==1){
66. pulse = (pulse + 5 > 10000) ? 10000 : pulse + 1; // increase by 1
67. } else {
68. pulse -= 5; // Decrease pulse by 5
69. if (pulse < 0) {
70. pulse = 0; // Clamp pulse to 0}}
71. if (light >= value) {
72. mode = 0;}
73. if (light < value) {
74. mode = 1;}
75. TIM1->CCR1 = pulse;}

```

2. When we shut down the LED and controlled only the shutter, we observed significant overshoot and large oscillations in the steady state. This occurred because our shutter had difficulties with small movements, leading to overshooting during both opening and closing.

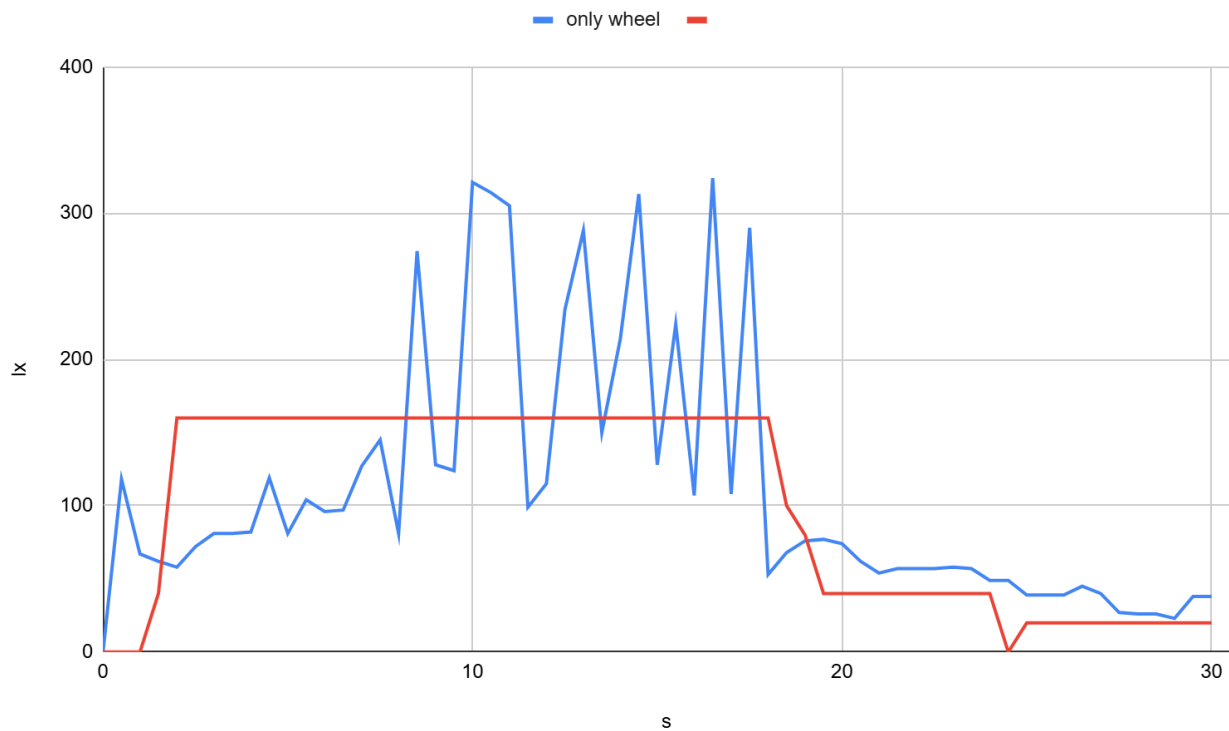


Diagram 3.

```

76. while (1){
77. HAL_Delay(200);
78. if (light >= 20+value) {
79.     engineR = 1; // Close
80. } else if(light < value-20){
81.     engineR = 2; // Open
82. }
83. else{
84.     engineR = 0;}
85. // Stepper motor movement with angle constraints
86. if (engineR == 1 && currentAngle <= maxAngle) {
87.     stepper_step_angle(STEPPER_STEP_SIZE, 0, 10);
88.     currentAngle += STEPPER_STEP_SIZE;
89. } else if (engineR == 2 && currentAngle >= minAngle) {
90.     stepper_step_angle(STEPPER_STEP_SIZE, 1, 10);
91.     currentAngle -= STEPPER_STEP_SIZE;
92. }}

```

3. When we controlled both the shutter and the LED, we achieved quite good gains and, at times, a stable steady state. This can be observed in the graph. Initially, the shutter overshoot and had some difficulty reaching a steady state. However, at the second set value, it achieved a good position, allowing the LED to take control and reach an almost perfect steady state.

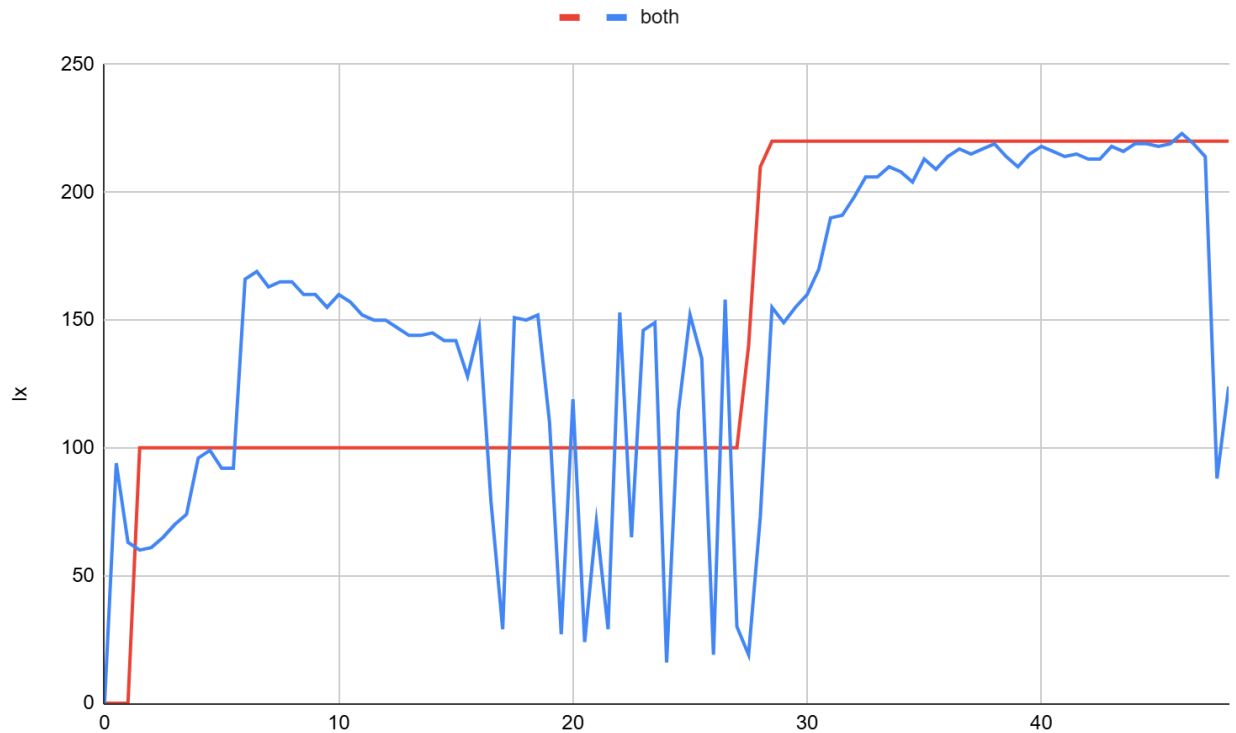


Diagram 4.

```

93. while (1){
94. HAL_Delay(200);
95. if (light >= 20+value) {
96.   engineR = 1; // Close
97. } else if(light < value-20){
98.   engineR = 2; // Open
99. }
100. else{
100.   engineR = 0;
101. }
101.   // Stepper motor movement with angle constraints
102.   if (engineR == 1 && currentAngle <= maxAngle) {
103.     stepper_step_angle(STEPPER_STEP_SIZE, 0, 10);
104.     currentAngle += STEPPER_STEP_SIZE;
105.   } else if (engineR == 2 && currentAngle >= minAngle) {
106.     stepper_step_angle(STEPPER_STEP_SIZE, 1, 10);
107.     currentAngle -= STEPPER_STEP_SIZE;
108.   }
109.   if (mode==1){
110.     pulse = (pulse + 5 > 10000) ? 10000 : pulse + 1; // increase by 1
111.   } else {

```

```
112.    pulse -= 5; // Decrease pulse by 5
113.    if (pulse < 0) {
114.        pulse = 0; // Clamp pulse to 0
115.    }
116.    }
117.    if (light >= value) {
118.        mode = 0;
119.    }
120.    if (light < value) {
121.        mode = 1;
122.    }
123.    TIM1->CCR1 = pulse;}
```

5. REFERENCES & SOURCES

1. Thingiverse mechanical iris 3D model by sketchpunk: <https://www.thingiverse.com/thing:773759>
2. We sincerely thank Mgr inż. Joanna Piasek-Skupna for her invaluable expertise and support.